

Cheaper per token, more expensive per task

Why downgrading the model in an agentic system often raises your LLM bill, and why every node that calls a model needs its own evaluation on accuracy, latency and cost.

TL;DR

- **Price per token is the wrong unit for an agent.** What you pay for is a *completed task*, and in an agentic loop those two numbers can move in opposite directions.
- **A weaker model at the decision-making node costs more turns, and every extra turn re-sends the whole conversation.** Cost grows faster than the number of turns. Add a lower success rate and a model that is 4× cheaper per token can end up 16% more expensive per successful task.
- **The same cheap model can be 75% cheaper somewhere else in the same system.** So the answer isn't "use bigger models". It's to measure every node that calls a model, on accuracy, latency and cost per completed task, and let the numbers assign the models.

The arithmetic is in `cost_model.py`. Run it, change the parameters, and check the claims against your own agent.

The first move everyone makes

When an AI feature's bill gets uncomfortable, the first lever everyone reaches for is the model. Swap the flagship for the smaller sibling, get a fraction of the per-token price, and ship the same feature for less money.

In an agent, the model you're most tempted to swap is the orchestrator: the one that reads the request, plans, chooses a tool, reads the result and decides what to do next. It runs on every turn of every conversation, so it's almost always the biggest line on the bill. On a cost dashboard it's the obvious first target.

It's also the node where downgrading most often backfires.

What happened when we tried it

We ran into this on an agent that works with an external tool through a set of tool calls, reading and writing records on the user's behalf. There was sensible pressure to default agents like this to cheaper models, and the obvious experiment was to run the same agent on a smaller model and compare.

Two things went wrong, and only one of them was the one we were watching for.

The expected one was quality. The smaller model got the tool-output mapping wrong. It attached the wrong titles and links to the records it returned, which is the kind of error that looks fine in a demo and breaks trust the first time a user clicks through. Moving back to a more capable model fixed it.

The unexpected one was cost. Doing the same job with the smaller model cost *more*. It didn't pick the right tool on the first try. It passed arguments that needed correcting, called things it didn't need, and took extra rounds to arrive where the capable model went directly. Each of those rounds was cheap. There were enough of them, and each was carrying enough context, that the total went up.

That second result is the one worth explaining. It isn't a quirk of one agent. It falls out of how agentic loops are billed.

Why a cheaper model can cost more

Three multipliers sit between the price per token and the price per task. A cheaper model at the decision-making node tends to make all three worse at once.

1. Turn inflation

A capable orchestrator tends to pick the right tool with the right arguments and move on. A weaker one explores: it picks a plausible but wrong tool, passes an argument in the wrong shape, reads an error, re-plans and tries again. Each misstep is at least one extra turn, and often two, since there's the wrong call and then the recovery.

2. Context growth, where the damage compounds

Chat-completion APIs are stateless. Every turn re-sends the entire conversation: the system prompt, every tool definition, the original request, and every call and result from earlier turns. So turn five doesn't cost what turn one did. It costs turn one plus everything accumulated since.

If the fixed part is *base* tokens and each turn adds about *inc*, input tokens over *T* turns are:

$$\text{input_tokens}(T) = T \cdot \text{base} + \text{inc} \cdot T(T-1)/2$$

The second term grows with the square of the number of turns. That's why turn inflation hurts more than it looks: in the example below, the cheaper model takes **2.33× as many turns but consumes 3.08× as many tokens**. The extra turns are also the most expensive ones, because they come last, when the context is longest.

3. Failure

Not every attempt completes the task. If a run fails and gets retried, the expected cost of one *successful* task is the cost of an attempt divided by the success rate. A weaker orchestrator fails more often, so its cost per attempt gets divided by a smaller number.

A worked example

Two configurations of the same orchestrator. The cheaper model is 4× cheaper per token, a typical gap between adjacent tiers of one model family. Everything below is produced by `cost_model.py`. These are illustrative parameters rather than measurements from any particular system, and they're there to make the mechanism concrete.

	Capable model	Cheaper model
Price per 1M tokens	\$2.00	\$0.50
Turns per attempt	3	7
Tokens per attempt	15,000	46,200
Cost per attempt	\$0.0300	\$0.0231
Success rate	90%	60%
Cost per successful task	\$0.0333	\$0.0385

Look at the two bold rows together, because the difference between them is the trap.

Per attempt, the cheaper model is cheaper: \$0.023 against \$0.030, a saving of about a quarter. That's what a cost dashboard shows you, because it counts calls and runs rather than outcomes. The swap looks like it worked.

Per successful task, the cheaper model is 16% more expensive. And that number leaves out the part of the bill that isn't tokens: the wrong links a user acted on, the support ticket that followed, and the engineer who spent an afternoon working out why.

The break-even rule

The example generalizes into one inequality. A cheaper model is actually cheaper per successful task only if:

$$\text{price ratio} > \text{token ratio} \times \text{success ratio}$$

where *price ratio* is how many times cheaper it is per token, *token ratio* is how many times more tokens it burns per attempt, and *success ratio* is the capable model's success rate divided by the cheaper one's.

Here the cheaper model is 4× cheaper, but it needs to be **4.62× cheaper** to break even (3.08×1.5). It falls short, so it loses.

The rule matters because it tells you exactly what you need to know before deciding: all three ratios. You can read one of them, the price ratio, off a pricing page. The other two only come from running the models on your task.

Where cheaper models do win

It would be easy to walk away from this with "always use the big model". That's wrong too, and the same arithmetic shows why.

Take a well-scoped leaf node: one call, a narrow job, a predictable output. Reformatting a tool result into a fixed schema, classifying a request into one of five intents, pulling three fields out of a known document type. There's no loop to inflate and no context to accumulate, and a smaller model often does the job about as reliably:

	Capable model	Cheaper model
Turns per attempt	1	1
Success rate	98%	97%
Cost per successful task	\$0.0086	\$0.0022

Same cheaper model, 75% cheaper. Here the price gap passes straight through, because nothing multiplies it away. Routing narrow, well-specified calls to smaller models is well established. FrugalGPT (Chen, Zaharia & Zou, 2023) showed that cascading queries across models of different cost can cut spend substantially while holding accuracy.

So the correct claim isn't that cheap models don't save money in agents. It's narrower and more useful:

A cheaper model does not automatically reduce the cost of an agentic system. At the node that makes decisions it often increases total cost. At well-scoped leaf nodes it often cuts cost substantially. You can't tell which case you're in without measuring cost per completed task, per node.

Evaluate every node, on three axes

If the right model depends on the node, the unit of evaluation has to be the node. Every place in the system where a model is called gets measured on its own terms:

Axis	Measured as	The failure it catches
Accuracy	Task success on a fixed evaluation set for that node, plus end-to-end success	A model that is cheap per call and wrong often enough to cost more overall
Latency	Wall-clock per completed task, not per call	Per-token speed swallowed by extra round trips
Cost	Dollars per <i>successful</i> task, including retries	The per-attempt saving that disappears per outcome

Latency behaves exactly like cost. A smaller model streams tokens faster, but seven round trips to a fast model can take longer than three to a slow one. The user waits for the task to finish, not for any single call.

What this looks like in practice:

Instrument every model call as a named node. Record the model, input and output tokens, latency and outcome for each call, tagged with which node made it. Without per-node attribution you get one blended cost line, and the orchestrator's turn inflation becomes invisible inside it.

Keep a fixed evaluation set per node, plus one end to end. The same inputs every time, so a change can be compared against the last one. A node-level set tells you whether the node still does its job. The end-to-end set tells you whether a cheaper node pushed the cost downstream. A "cheap" leaf that returns slightly malformed output can make the orchestrator retry, and the saving shows up on one line while the bill rises on another.

Start capable and optimize downward. Get the system working on the most capable model first, so you have a baseline that reflects what's possible rather than what's cheap. Then try cheaper models one node at a time, and keep a downgrade only if end-to-end cost per successful task and quality both hold. Going the other way, starting cheap and upgrading whatever visibly breaks, only catches failures that are loud. Turn inflation is quiet.

Re-run the evaluation when anything changes. A new model version, a new tool, a revised prompt or a price change can each move all three ratios. The evaluation set is what lets you answer "should we switch?" in an afternoon rather than by argument.

The architecture that falls out

Measure honestly and a pattern tends to appear. It runs against the cost-cutting instinct:

- **Put capable models where decisions are made:** the orchestrator, the planner, the router. Errors there compound through every turn that follows.
- **Put cheap models where the work is well specified:** extraction, classification, formatting, summarizing a known structure. There the price gap passes through intact.

The instinct says the orchestrator should be downgraded first, because it's the biggest line on the bill. The arithmetic says it's usually the last place to downgrade, because it's the one node whose mistakes multiply.

Closing

"Which model is cheapest?" is the wrong question for an agent. The useful one is *which model is cheapest per task done well, at this node?* A pricing page can't answer that. The only thing that can is an evaluation that measures accuracy, latency and cost per completed task, run against your own system.

The price per token is the one number in this whole calculation you can read off a web page, and it's the one that matters least.

I build AI agents and the evaluation systems that decide how they're configured. Reproduce the numbers in this piece with `cost_model.py`.

Reference: Lingjiao Chen, Matei Zaharia, James Zou. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*. arXiv:2305.05176, 2023.